

TECHNICAL ARCHITECTURE — GRAND VISION

Technical Architecture

Grand Vision

v2.0 · Richard Uzor — May 2026 · Devstrike Digital Limited

01 — ARCHITECTURE OVERVIEW

WaMPaS is a modern, decoupled web application built in three converging layers. The architecture is designed to start simple and scale by layer — no premature complexity. Each phase introduces new architectural components only when the product and traffic demand them.

02 — TECHNOLOGY STACK

Layer	Technology	Rationale
Frontend	Next.js 14 (App Router) + TypeScript	SSR for SEO on all archive/event pages, React Server Components, PWA-capable
Styling	Tailwind CSS + DM Serif Display + Inter	Design system tokens, mobile-first, Google Fonts
State management	Zustand (client) + TanStack Query (server)	Lightweight global state, powerful data fetching with caching

Layer	Technology	Rationale
Forms	React Hook Form + Zod	Type-safe validation, consistent error UX
Auth	NextAuth.js v5	JWT sessions, Google OAuth for calendar, extensible for SSO
API	Node.js + TypeScript + Fastify	High-performance REST API, type-safe, Zod validation
Real-time (Phase 3)	WebSockets via Ably or self-hosted Soketi	Fellowship rooms, prayer rooms live sessions, DMs
Database	PostgreSQL 15 + Prisma ORM	Relational integrity, Prisma migrations, type-safe queries
Cache	Upstash Redis	Rate limiting, session cache, hot feed data, reaction counts
Search (Phase 1-2)	PostgreSQL full-text (tsvector + GIN)	Zero additional infra for Phase 1 scale
Search (Phase 3)	pgvector + OpenAI embeddings	Semantic search across archive — 'find everything said about fear'
Storage	Cloudflare R2	Zero egress fees, fast CDN delivery to Nigeria
Email	Resend + React Email	Transactional email with branded templates
Payments	Paystack + Flutterwave	NGN primary, diaspora currency fallback
Deployment	Vercel (frontend) + Railway (API + DB)	Zero-config CI/CD, auto-scaling, managed Postgres

03 — ROUTE ARCHITECTURE

Route	Render	Layer	Notes
/	SSR	Archive	Message feed — landing page, SEO-indexed
	SSR	Archive	

Route	Render	Layer	Notes
/messages/[id]			Message detail — every message is a unique indexed page
/pastors/[id]	SSR	Archive	Pastor profile — indexed
/churches/[id]	SSR	Archive	Church profile — indexed
/events/[id]	SSR	Events	Event detail — indexed
/feed	SSR+CSR	Social	Public feed — SSR first paint, CSR pagination
/rooms/[id]	CSR	Social	Fellowship room — real-time, not indexed
/prayer/[id]	CSR	Social	Prayer room — real-time, not indexed
/search	CSR	All	Search — dynamic, not indexed
/profile	CSR	All	Authenticated user profile
/admin	CSR	Admin	Protected route, separate subdomain in Phase 3

04 — DATABASE SCHEMA SUMMARY

Table	Key Fields	Relations
users	id, name, email, password_hash, role, church_id	has_many messages, saved_messages, room_memberships
messages	id, text, date_preached, pastor_id, church_id, event_id, category[]	belongs_to pastor, church, event, user
pastors	id, title, name, church_id, bio, image_url, is_verified	has_many messages, events, room_appearances
churches	id, name, denomination, address, city, is_verified	has_many pastors, messages, events, rooms
events	id, name, type, hosting_church_id, start_date, is_sponsored	has_many messages, testimonies, rooms

Table	Key Fields	Relations
rooms	id, name, type, church_id, owner_id, privacy	has_many memberships, posts, prayer_requests
prayer_requests	id, room_id, user_id, text, is_anonymous, is_answered	belongs_to room; has_many intercessions, testimony
feed_posts	id, user_id, type, text, archive_message_id, image_url	belongs_to user; optionally links to archive message
reactions	id, target_type, target_id, type, user_id	Polymorphic: message, feed_post, room_post
payments	id, user_id, type, amount, status, paystack_ref	Polymorphic: ad, subscription, donation, verification

05 — SECURITY ARCHITECTURE

Concern	Mechanism
Authentication	JWT in httpOnly cookies. XSS cannot read tokens. SameSite=Strict.
Input validation	Zod schemas on all API inputs. HTML sanitised with DOMPurify.
SQL injection	Prisma parameterised queries throughout.
Rate limiting	Per-IP and per-user via Redis. Auth endpoints: 5/min with 15-min lockout.
File uploads	Magic byte type validation. Sharp strips EXIF. 5MB limit.
Webhook security	Paystack: HMAC-SHA512 signature verification.
Real-time auth (Phase 3)	WebSocket connections authenticated via JWT token on handshake.
Secrets	Railway environment variables. Never in source code.

